



I304 - Arquitectura de computadoras y Sistemas Operativos

Trabajo Práctico 2: Bomba Binaria

Lucas Ercolano

20 de abril de 2025

Ingeniería en Inteligencia Artificial

Fase 1

Para superar la primera fase, se debe comprender que el programa efectúa una comparación entre dos cadenas de texto. Una de estas cadenas corresponde a la entrada proporcionada por el usuario, cuyo puntero se almacena en el registro `rdi`. La otra es una cadena específica predefinida por el programa, cuyo puntero se carga en el registro `rsi` durante la ejecución de la función `phase_1`.

La verificación consiste en comprobar si ambas cadenas poseen idéntica longitud y, posteriormente, si coinciden carácter por carácter. Si las cadenas no son exactamente iguales, se produce la detonación de la bomba.

El objetivo para desactivar esta fase es descubrir la cadena predefinida. Esto se logra mediante el uso de un depurador (como `gdb`) para analizar la ejecución de la función `phase_1`. Se debe identificar la instrucción que asigna la dirección de memoria de la cadena predefinida al registro `rsi`.

Una vez obtenida dicha dirección, se puede examinar el contenido de la memoria en esa ubicación para determinar la cadena de texto. Esta cadena constituye la entrada correcta que debe proporcionarse para que la comparación resulte exitosa y se pueda avanzar a la siguiente fase.

Fase 2

La segunda fase del programa requiere el procesamiento de dos números enteros, denominados aquí n_1 y n_2 , que se extraen de la segunda línea del archivo de entrada. Internamente, se utiliza la función `strtol` para convertir las cadenas de texto de dicha línea en estos dos valores numéricos enteros.

Para desactivar esta fase y evitar la detonación, se deben cumplir simultáneamente dos condiciones específicas sobre estos números:

1. **Conteo de Bits Específico:** Se realiza un cálculo intermedio: $\text{resultado} = n_1 + n_2 - 32$. La representación binaria de 32 bits de este **resultado** debe contener exactamente once bits con valor 1 (bits activos o 'set bits'). Esta verificación se lleva a cabo dentro de una sub-función (referida como `misterio`), la cual implementa un algoritmo de conteo que itera 32 veces, utilizando desplazamientos aritméticos a la derecha (`sar`) y operaciones lógicas AND con el valor 1 para aislar y sumar cada bit. Al finalizar el bucle de conteo, se compara el total acumulado con el valor 11. Si no son iguales, una instrucción de salto condicional (`jne`) redirige el flujo para detonar la bomba.
2. **Signo del Resultado XOR:** Se efectúa una operación XOR a nivel de bits entre los dos números de entrada originales: $\text{resultado_xor} = n_1 \oplus n_2$ (donde $\oplus$ representa la operación XOR). El **resultado_xor**, interpretado como un número entero con signo de 32 bits, debe ser negativo. Esto equivale a verificar que el bit más significativo (MSB) del **resultado_xor** sea 1. Una instrucción de salto condicional basada en el signo (`js`, Jump if Sign) evalúa esta condición; si el resultado no es negativo, se procede a detonar la bomba.

Fase 3

La tercera fase del programa requiere como entrada una cadena de texto (una palabra) y un número entero, los cuales se leen desde la tercera línea del archivo de entrada. La función `sscanf` se utiliza internamente con el formato `"%s%d"` para parsear estos dos elementos desde la línea de entrada. Una entrada que no se ajuste a este formato específico provocará la detonación inmediata de la bomba.

Para desactivar la fase 3 de manera exitosa, se deben cumplir simultáneamente las siguientes condiciones:

1. **Existencia de la Palabra:** La palabra proporcionada como primer elemento de la entrada debe ser una de las palabras válidas listadas en el archivo externo `palabras.txt`.
2. **Profundidad de Búsqueda dentro de un Rango:** Una función interna busca la palabra ingresada dentro de la lista contenida en `palabras.txt`. Esta búsqueda se realiza mediante un algoritmo de búsqueda binaria recursiva. La cantidad de llamadas recursivas efectuadas hasta encontrar la palabra (es decir, la profundidad de la recursión) es un valor crucial. Para superar la fase, esta profundidad debe ser estrictamente mayor que 6 y estrictamente menor que 11. En otras palabras, la profundidad válida debe ser 7, 8, 9 o 10. El código contiene verificaciones explícitas (mediante comparaciones y saltos condicionales como `jg` o `j1`) para asegurar que la profundidad (cuyo valor se almacena temporalmente, por ejemplo, en el registro `eax`) caiga dentro de este intervalo permitido.
3. **Coincidencia del Número con la Profundidad:** El número entero proporcionado como segundo elemento de la entrada debe ser exactamente igual a la profundidad de recursión determinada por la función `cuenta` al encontrar la palabra ingresada. Al finalizar la ejecución de `cuenta`, se realizan comparaciones adicionales para asegurar que el número ingresado coincida con la profundidad calculada y que esta profundidad cumpla con el límite inferior (ser mayor que 6).

Fase 4

El proceso de resolución de la Fase 4 se inicia mediante el análisis estático y dinámico del código ensamblador correspondiente a la función `phase_4`.

Se observa que la función primero determina la longitud de la cadena de caracteres proporcionada como entrada. Esto se realiza mediante una llamada a la función `string_length`. El resultado de esta llamada (la longitud de la cadena) se compara inmediatamente con el valor constante 6. Se requiere que la cadena de entrada tenga *exactamente 6 caracteres*. Si la longitud difiere de 6, la ejecución se desvía hacia la función `explode_bomb`, finalizando el programa de forma prematura.

Si la validación de longitud es exitosa, la ejecución procede a un bucle diseñado para iterar sobre cada uno de los 6 caracteres de la cadena de entrada. Dentro de este bucle, se ejecuta la siguiente lógica para cada carácter:

- Se obtiene el valor ASCII del carácter actual.

- Se realiza una operación AND bit a bit entre dicho valor ASCII y la máscara hexadecimal 0xF (equivalente a 15 en decimal). Esta operación aísla los 4 bits menos significativos del valor ASCII, produciendo un resultado numérico en el rango de 0 a 15.
- Este resultado numérico se utiliza como índice para acceder a un arreglo de datos predefinido en memoria. La dirección base de este arreglo, según el análisis del código, es 0x4cde60.
- El valor entero recuperado del arreglo en la posición indicada por el índice calculado se suma a un registro acumulador (específicamente, `ecx`), el cual fue inicializado a cero antes de comenzar el bucle.

Este proceso se repite para los 6 caracteres de la cadena de entrada.

Una vez que el bucle ha procesado los 6 caracteres, se realiza una comparación final. El valor acumulado en el registro `ecx` se compara con la constante hexadecimal 0x2e (equivalente a 46 en decimal). Si el valor acumulado *no es igual* a 46, la ejecución salta a la función `explode_bomb`. Para desactivar la fase satisfactoriamente, es imperativo que la suma final de los 6 valores extraídos del arreglo sea exactamente 46.

Basado en el análisis anterior, se sigue un procedimiento para encontrar la cadena de entrada correcta.

El primer paso es determinar los valores contenidos en el arreglo ubicado en 0x4cde60. Se utiliza el depurador GDB mientras se ejecuta la bomba. El comando `x/16dw 0x4cde60` permite examinar los primeros 16 elementos del arreglo, mostrando cada valor como un entero decimal de 4 bytes (DWORD). Esto proporciona la correspondencia entre cada índice (0-15) y el valor numérico almacenado.

Con los valores del arreglo conocidos, el siguiente paso es encontrar una combinación de 6 de estos valores cuya suma total sea precisamente 46. Es crucial recordar los índices (0-15) asociados a los valores seleccionados. Pueden existir múltiples combinaciones válidas. Los índices correspondientes a estos valores, según el arreglo inspeccionado, son 14, 7, 13, 4, 8, 8.

El último paso consiste en encontrar 6 caracteres tales que, para cada uno, el resultado de la operación `valor_ascii & 15` coincida con uno de los índices de la combinación encontrada (en el ejemplo, 14, 7, 13, 4, 8, 8). Es importante notar que varios caracteres pueden producir el mismo índice después de aplicar la máscara. Por ejemplo, tanto 'H' (ASCII 72) como 'X' (ASCII 88) resultan en 8 al aplicar `& 15`. Por lo tanto, pueden existir múltiples cadenas de caracteres que constituyan una solución válida. Siguiendo con el ejemplo de índices 14, 7, 13, 4, 8, 8, una posible selección de caracteres podría ser 'N' ($78 \& 15 = 14$), 'G' ($71 \& 15 = 7$), 'M' ($77 \& 15 = 13$), 'D' ($68 \& 15 = 4$), 'H' ($72 \& 15 = 8$), 'H' ($72 \& 15 = 8$).

Fase 5

Esta fase, comúnmente referida como la "fase secreta", no forma parte de la secuencia estándar de desactivación y requiere una activación específica.

La existencia de esta fase se descubre típicamente al analizar la función `phase_defused`, la cual se ejecuta tras completar exitosamente otras fases. La activación de la fase secreta depende de una modificación en la entrada proporcionada para la **Fase 3**. Específicamente, se debe añadir un **tercer argumento** a la línea de entrada de la Fase 3. El formato esperado para esta

línea de entrada, detectado mediante funciones como `sscanf`, es del tipo `"%s%d%s"`. Crucialmente, este tercer argumento debe ser exactamente la cadena de texto `.abrete_sesamo`. Una comparación de cadenas (`string_compare`) verifica que el tercer argumento proporcionado coincida con esta clave secreta.

Una vez activada, la fase secreta opera sobre una estructura de datos interna que corresponde a un Árbol Binario de Búsqueda (BST), el cual está predefinido en la memoria del programa. La lógica principal reside en una función recursiva (identificada como `fun7` en análisis previos) que navega este árbol.

Esta función recursiva toma dos parámetros principales: un puntero al nodo actual que se está visitando en el árbol y el **número entero** que se proporcionó como entrada numérica para *esta fase secreta*. Es importante notar que este número es independiente de los argumentos originales de la Fase 3, aparte de la cadena de activación.

La navegación a través del árbol y el cálculo del valor resultante se rigen por las siguientes reglas, basadas en la comparación del número de entrada con el valor almacenado en el nodo actual:

- **Caso Base (Coincidencia):** Si el número de entrada es igual al valor del nodo actual, la función recursiva retorna inmediatamente el valor 0.
- **Recursión Izquierda:** Si el número de entrada es menor que el valor del nodo actual, la función se invoca recursivamente sobre el hijo izquierdo. Si res_{izq} es el valor devuelto por esta llamada recursiva, el valor retornado por la llamada actual será $2 \times res_{izq}$.
- **Recursión Derecha:** Si el número de entrada es mayor que el valor del nodo actual, la función se invoca recursivamente sobre el hijo derecho. Si res_{der} es el valor devuelto por esta llamada recursiva, el valor retornado por la llamada actual será $2 \times res_{der} + 1$.

El objetivo de esta fase es encontrar un número entero que, al ser procesado por la función recursiva comenzando desde la raíz del BST, produzca un valor final específico al retornar la llamada inicial.